



AHDS - PIC ASSEMBLY FOR EMBEDDED & HARDWARE ENGINEERS

***PIC16 • PIC18 • Registers • Memory • Interrupts • Timers • ADC • PWM • UART • SPI • I²C • EEPROM • MPLAB X • PIC-AS**
500 Theory, Calculation & Practical Questions — Français / English

A low-level study guide: architecture → instruction set → timing → peripherals → debugging → legacy MPASM migration.

How to use this document / Comment utiliser ce document

FR : Ce questionnaire vise surtout les PIC 8 bits classiques et modernes, notamment les familles PIC16 et PIC18. Toujours vérifier le datasheet exact du composant : registres, banques, vecteurs, périphériques et détails d'instructions varient selon le cœur et le device.

EN: This questionnaire focuses mainly on classic and modern 8-bit PIC devices, especially PIC16 and PIC18 families. Always verify the exact device datasheet: registers, banks, vectors, peripherals, and instruction details vary by core and device.

FR : Les questions de calcul utilisent fréquemment Fosc/4 comme horloge d'instruction, caractéristique des PIC 8 bits classiques. Les branches, appels, retours et instructions de saut/skip peuvent demander des cycles supplémentaires selon le cœur.

EN: Calculation questions frequently use Fosc/4 as the instruction clock, characteristic of classic 8-bit PIC cores. Branches, calls, returns, and skip/branch instructions can require extra cycles depending on the core.

FR : Le document couvre le legacy MPASM, mais privilégie également l'écosystème actuel MPLAB X + MPLAB XC8 PIC Assembler (pic-as).

EN: The document covers legacy MPASM while also emphasizing the current MPLAB X + MPLAB XC8 PIC Assembler (pic-as) ecosystem.

Core formulas / Formules essentielles

Concept	Formula / Formule
Instruction frequency	$F_{cy} = F_{osc} / 4$ (classic 8-bit PIC cores)
Instruction period	$T_{cy} = 1 / F_{cy} = 4 / F_{osc}$
Loop delay	$T_{delay} \approx \Sigma(\text{instruction cycles}) \times T_{cy}$
Timer tick	$Tick = \text{prescaler} / F_{timer}$
Timer overflow	$T_{overflow} = \text{counts_to_overflow} \times Tick$
UART ideal bit time	$T_{bit} = 1 / \text{Baud}$
PWM duty	$Duty \approx T_{on} / T_{period} \times 100\%$
ADC LSB ideal	$LSB \approx V_{ref} / (2^N - 1)$ or $V_{ref}/2^N$ depending on convention
ADC voltage estimate	$V_{in} \approx Code \times V_{ref} / FullScaleCode$
I ² C rise-time concept	$t_r \approx 0.8473 \times R_p \times C_{bus}$ for a first-order RC 30–70% approximation

1. PIC Architecture Fundamentals / Architecture PIC

#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
1	Q: Qu'est-ce que Harvard architecture ? R: Des espaces/bus séparés pour mémoire programme et données, principe architectural des pic classiques.	Q: What is Harvard architecture? A: Separate program and data memory spaces/buses, a defining architectural idea of classic pic mcus.
2	Q: Pourquoi Harvard architecture est-il important en programmation PIC Assembly ? R: Des espaces/bus séparés pour mémoire programme et données, principe architectural des pic classiques. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is Harvard architecture important in PIC Assembly programming? A: Separate program and data memory spaces/buses, a defining architectural idea of classic pic mcus. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
3	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec Harvard architecture ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for Harvard architecture? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
4	Q: Comment valider Harvard architecture sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate Harvard architecture on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
5	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de Harvard architecture. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving Harvard architecture. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
6	Q: Qu'est-ce que instruction pipeline ? R: Le chevauchement fetch/exécution permettant à de nombreuses instructions de terminer au rythme du cycle d'instruction.	Q: What is instruction pipeline? A: Overlap of instruction fetch and execution that helps many instructions complete at the instruction-cycle rate.
7	Q: Pourquoi instruction pipeline est-il important en programmation PIC Assembly ? R: Le chevauchement fetch/exécution permettant à de nombreuses instructions de terminer au rythme du cycle d'instruction. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is instruction pipeline important in PIC Assembly programming? A: Overlap of instruction fetch and execution that helps many instructions complete at the instruction-cycle rate. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
8	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec instruction pipeline ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for instruction pipeline? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
9	Q: Comment valider instruction pipeline sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate instruction pipeline on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
10	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de instruction pipeline. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving instruction pipeline. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
11	Q: Qu'est-ce que Fosc and Fcy ? R: La fréquence oscillateur système et la fréquence d'instruction dérivée; sur pic 8 bits classiques fcy vaut couramment fosc/4.	Q: What is Fosc and Fcy? A: System oscillator frequency and derived instruction frequency; on classic 8-bit pic cores fcy is commonly fosc/4.
12	Q: Pourquoi Fosc and Fcy est-il important en programmation PIC Assembly ? R: La fréquence oscillateur système et la fréquence d'instruction dérivée; sur pic 8 bits classiques fcy vaut couramment fosc/4. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is Fosc and Fcy important in PIC Assembly programming? A: System oscillator frequency and derived instruction frequency; on classic 8-bit pic cores fcy is commonly fosc/4. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
13	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec Fosc and Fcy ? R: Pour un PIC 8 bits classique à Fosc=20 MHz : Fcy=5 MHz et Tcy=200 ns. Vérifier le cœur/device exact avant d'appliquer la règle.	Q: What calculation or practical check should you know for Fosc and Fcy? A: For a classic 8-bit PIC at Fosc=20 MHz: Fcy=5 MHz and Tcy=200 ns. Verify the exact core/device before applying the rule.
14	Q: Comment valider Fosc and Fcy sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate Fosc and Fcy on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
15	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de Fosc and Fcy. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving Fosc and Fcy. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
16	Q: Qu'est-ce que PIC16 core ? R: Une famille d'architectures pic 8 bits mid-range/enhanced-mid-range avec caractéristiques dépendant du device.	Q: What is PIC16 core? A: A mid-range/enhanced-mid-range 8-bit pic architecture family with device-dependent instruction and memory features.
17	Q: Pourquoi PIC16 core est-il important en programmation PIC Assembly ? R: Une famille d'architectures pic 8 bits mid-range/enhanced-mid-range avec caractéristiques dépendant du device. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is PIC16 core important in PIC Assembly programming? A: A mid-range/enhanced-mid-range 8-bit pic architecture family with device-dependent instruction and memory features. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
18	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec PIC16 core ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for PIC16 core? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
19	Q: Comment valider PIC16 core sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate PIC16 core on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
20	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de PIC16 core. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving PIC16 core. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
21	Q: Qu'est-ce que PIC18 core ? R: Une architecture pic 8 bits avec jeu d'instructions et mécanismes mémoire plus riches que les pic16 mid-range classiques.	Q: What is PIC18 core? A: An 8-bit pic architecture with a richer instruction set and memory/access mechanisms than classic mid-range pic16 cores.
22	Q: Pourquoi PIC18 core est-il important en programmation PIC Assembly ? R: Une architecture pic 8 bits avec jeu d'instructions et mécanismes mémoire plus riches que les pic16 mid-range classiques. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is PIC18 core important in PIC Assembly programming? A: An 8-bit pic architecture with a richer instruction set and memory/access mechanisms than classic mid-range pic16 cores. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
23	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec PIC18 core ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for PIC18 core? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
24	Q: Comment valider PIC18 core sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate PIC18 core on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
25	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de PIC18 core. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving PIC18 core. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.

2. CPU Registers & Status / Registres CPU

#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
26	Q: Qu'est-ce que W/WREG ? R: Le registre de travail utilisé comme opérande/résultat central dans de nombreuses instructions.	Q: What is W/WREG? A: The working register used as a central operand/result register in many pic instructions.
27	Q: Pourquoi W/WREG est-il important en programmation PIC Assembly ? R: Le registre de travail utilisé comme opérande/résultat central dans de nombreuses instructions. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is W/WREG important in PIC Assembly programming? A: The working register used as a central operand/result register in many pic instructions. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
28	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec WWREG ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for WWREG? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
29	Q: Comment valider WWREG sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate WWREG on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
30	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de WWREG. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving WWREG. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
31	Q: Qu'est-ce que STATUS register ? R: Un registre d'état/contrôle contenant flags arithmétiques et, sur certains cœurs, informations de sélection de banque.	Q: What is STATUS register? A: A status/control register containing arithmetic flags and, on some cores, memory-bank selection information.
32	Q: Pourquoi STATUS register est-il important en programmation PIC Assembly ? R: Un registre d'état/contrôle contenant flags arithmétiques et, sur certains cœurs, informations de sélection de banque. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is STATUS register important in PIC Assembly programming? A: A status/control register containing arithmetic flags and, on some cores, memory-bank selection information. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
33	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec STATUS register ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for STATUS register? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
34	Q: Comment valider STATUS register sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate STATUS register on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
35	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de STATUS register. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving STATUS register. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
36	Q: Qu'est-ce que zero flag ? R: Un flag indiquant un résultat nul pour les instructions qui le modifient.	Q: What is zero flag? A: A status flag indicating a zero result for instructions that affect it.
37	Q: Pourquoi zero flag est-il important en programmation PIC Assembly ? R: Un flag indiquant un résultat nul pour les instructions qui le modifient. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is zero flag important in PIC Assembly programming? A: A status flag indicating a zero result for instructions that affect it. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
38	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec zero flag ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for zero flag? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
39	Q: Comment valider zero flag sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate zero flag on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
40	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de zero flag. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving zero flag. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
41	Q: Qu'est-ce que carry flag ? R: Un flag utilisé par opérations arithmétiques et rotations, avec sémantique définie par le jeu d'instructions.	Q: What is carry flag? A: A status flag used by arithmetic and rotate operations, with exact semantics defined by the instruction set.
42	Q: Pourquoi carry flag est-il important en programmation PIC Assembly ? R: Un flag utilisé par opérations arithmétiques et rotations, avec sémantique définie par le jeu d'instructions. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is carry flag important in PIC Assembly programming? A: A status flag used by arithmetic and rotate operations, with exact semantics defined by the instruction set. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
43	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec carry flag ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for carry flag? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
44	Q: Comment valider carry flag sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate carry flag on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
45	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de carry flag. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving carry flag. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
46	Q: Qu'est-ce que program counter ? R: Le registre/état sélectionnant l'adresse de la prochaine instruction, avec largeur et pagination dépendant du device.	Q: What is program counter? A: The register/state selecting the next program-memory instruction address, with device-specific width and paging behavior.
47	Q: Pourquoi program counter est-il important en programmation PIC Assembly ? R: Le registre/état sélectionnant l'adresse de la prochaine instruction, avec largeur et pagination dépendant du device. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is program counter important in PIC Assembly programming? A: The register/state selecting the next program-memory instruction address, with device-specific width and paging behavior. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
48	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec program counter ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for program counter? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
49	Q: Comment valider program counter sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate program counter on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
50	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de program counter. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving program counter. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.

3. Program & Data Memory / Mémoire programme et données

#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
51	Q: Qu'est-ce que program memory ? R: La mémoire non volatile des instructions, généralement flash sur les pic modernes.	Q: What is program memory? A: Nonvolatile instruction storage, typically flash on modern pic mcus.
52	Q: Pourquoi program memory est-il important en programmation PIC Assembly ? R: La mémoire non volatile des instructions, généralement flash sur les pic modernes. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is program memory important in PIC Assembly programming? A: Nonvolatile instruction storage, typically flash on modern pic mcus. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
53	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec program memory ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for program memory? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
54	Q: Comment valider program memory sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate program memory on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
55	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de program memory. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving program memory. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
56	Q: Qu'est-ce que file register ? R: Le terme pic pour registres adressables en mémoire de données, incluant sfr et gpr.	Q: What is file register? A: The pic term for addressable data-memory registers, including sfrs and gprs.
57	Q: Pourquoi file register est-il important en programmation PIC Assembly ? R: Le terme pic pour registres adressables en mémoire de données, incluant sfr et gpr. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is file register important in PIC Assembly programming? A: The pic term for addressable data-memory registers, including sfrs and gprs. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
58	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec file register ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for file register? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
59	Q: Comment valider file register sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate file register on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
60	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de file register. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving file register. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
61	Q: Qu'est-ce que SFR ? R: Special function register contrôlant cpu ou périphériques.	Q: What is SFR? A: Special function register controlling cpu or peripheral hardware.
62	Q: Pourquoi SFR est-il important en programmation PIC Assembly ? R: Special function register contrôlant cpu ou périphériques. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is SFR important in PIC Assembly programming? A: Special function register controlling cpu or peripheral hardware. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
63	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec SFR ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for SFR? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
64	Q: Comment valider SFR sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate SFR on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
65	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de SFR. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving SFR. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
66	Q: Qu'est-ce que GPR ? R: General purpose register utilisé pour variables/données logicielles.	Q: What is GPR? A: General purpose register used for software variables/data.
67	Q: Pourquoi GPR est-il important en programmation PIC Assembly ? R: General purpose register utilisé pour variables/données logicielles. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is GPR important in PIC Assembly programming? A: General purpose register used for software variables/data. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
68	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec GPR ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for GPR? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
69	Q: Comment valider GPR sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate GPR on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
70	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de GPR. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving GPR. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
71	Q: Qu'est-ce que banked memory ? R: Une organisation mémoire divisée en banques exigeant sélection ou mécanismes d'accès selon le cœur.	Q: What is banked memory? A: Data-memory organization divided into banks requiring bank selection or access mechanisms on relevant cores.
72	Q: Pourquoi banked memory est-il important en programmation PIC Assembly ? R: Une organisation mémoire divisée en banques exigeant sélection ou mécanismes d'accès selon le cœur. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is banked memory important in PIC Assembly programming? A: Data-memory organization divided into banks requiring bank selection or access mechanisms on relevant cores. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
73	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec banked memory ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for banked memory? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
74	Q: Comment valider banked memory sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate banked memory on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
75	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de banked memory. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving banked memory. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.

4. Addressing, Banking & Indirect Access / Adressage

#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
76	Q: Qu'est-ce que bank selection ? R: La sélection de la banque mémoire contenant le registre cible lorsque l'adressage direct est banké.	Q: What is bank selection? A: Selection of the data-memory bank containing a target register on devices where direct addressing is banked.
77	Q: Pourquoi bank selection est-il important en programmation PIC Assembly ? R: La sélection de la banque mémoire contenant le registre cible lorsque l'adressage direct est banké. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is bank selection important in PIC Assembly programming? A: Selection of the data-memory bank containing a target register on devices where direct addressing is banked. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
78	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec bank selection ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for bank selection? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
79	Q: Comment valider bank selection sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate bank selection on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
80	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de bank selection. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving bank selection. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
81	Q: Qu'est-ce que BSR ? R: Bank select register utilisé par pic18 et certaines architectures améliorées pour sélectionner les banques.	Q: What is BSR? A: Bank select register used by pic18 and some enhanced architectures to select data-memory banks.
82	Q: Pourquoi BSR est-il important en programmation PIC Assembly ? R: Bank select register utilisé par pic18 et certaines architectures améliorées pour sélectionner les banques. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is BSR important in PIC Assembly programming? A: Bank select register used by pic18 and some enhanced architectures to select data-memory banks. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
83	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec BSR ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for BSR? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
84	Q: Comment valider BSR sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate BSR on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
85	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de BSR. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving BSR. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
86	Q: Qu'est-ce que access bank ? R: Un mécanisme pic18 permettant l'accès à certaines zones sans modifier bsr.	Q: What is access bank? A: A pic18 mechanism allowing selected low/high data-memory regions to be accessed without changing bsr.
87	Q: Pourquoi access bank est-il important en programmation PIC Assembly ? R: Un mécanisme pic18 permettant l'accès à certaines zones sans modifier bsr. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is access bank important in PIC Assembly programming? A: A pic18 mechanism allowing selected low/high data-memory regions to be accessed without changing bsr. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
88	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec access bank ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for access bank? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
89	Q: Comment valider access bank sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate access bank on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
90	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de access bank. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving access bank. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
91	Q: Qu'est-ce que FSR ? R: File select register servant de pointeur pour l'accès indirect.	Q: What is FSR? A: File select register used as an address pointer for indirect data-memory access.
92	Q: Pourquoi FSR est-il important en programmation PIC Assembly ? R: File select register servant de pointeur pour l'accès indirect. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is FSR important in PIC Assembly programming? A: File select register used as an address pointer for indirect data-memory access. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
93	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec FSR ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for FSR? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
94	Q: Comment valider FSR sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate FSR on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
95	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de FSR. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving FSR. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
96	Q: Qu'est-ce que INDF/indirect access ? R: Un mécanisme accédant aux données via un pointeur fsr plutôt qu'une adresse directe.	Q: What is INDF/indirect access? A: A mechanism that accesses data through an fsr pointer rather than a literal file-register address.
97	Q: Pourquoi INDF/indirect access est-il important en programmation PIC Assembly ? R: Un mécanisme accédant aux données via un pointeur fsr plutôt qu'une adresse directe. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is INDF/indirect access important in PIC Assembly programming? A: A mechanism that accesses data through an fsr pointer rather than a literal file-register address. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
98	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec INDF/indirect access ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for INDF/indirect access? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
99	Q: Comment valider INDF/indirect access sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate INDF/indirect access on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
100	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de INDF/indirect access. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving INDF/indirect access. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.

5. Core Data-Movement Instructions / Instructions de transfert

#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
101	Q: Qu'est-ce que MOVLW ? R: Charger une valeur littérale dans w/wreg.	Q: What is MOVLW? A: Move a literal value into w/wreg.
102	Q: Pourquoi MOVLW est-il important en programmation PIC Assembly ? R: Charger une valeur littérale dans w/wreg. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is MOVLW important in PIC Assembly programming? A: Move a literal value into w/wreg. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
103	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec MOVLW ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for MOVLW? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
104	Q: Comment valider MOVLW sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate MOVLW on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
105	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de MOVLW. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving MOVLW. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
106	Q: Qu'est-ce que MOVWF ? R: Transférer w/wreg vers un file register.	Q: What is MOVWF? A: Move w/wreg into a file register.
107	Q: Pourquoi MOVWF est-il important en programmation PIC Assembly ? R: Transférer w/wreg vers un file register. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is MOVWF important in PIC Assembly programming? A: Move w/wreg into a file register. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
108	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec MOVWF ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for MOVWF? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
109	Q: Comment valider MOVWF sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate MOVWF on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
110	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de MOVWF. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving MOVWF. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
111	Q: Qu'est-ce que MOVF ? R: Lire/transférer la valeur d'un file register vers une destination en modifiant les flags définis.	Q: What is MOVF? A: Move/read a file-register value to a selected destination while affecting defined flags.
112	Q: Pourquoi MOVF est-il important en programmation PIC Assembly ? R: Lire/transférer la valeur d'un file register vers une destination en modifiant les flags définis. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is MOVF important in PIC Assembly programming? A: Move/read a file-register value to a selected destination while affecting defined flags. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
113	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec MOVF ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for MOVF? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
114	Q: Comment valider MOVF sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate MOVF on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
115	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de MOVF. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving MOVF. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
116	Q: Qu'est-ce que CLRF ? R: Mettre un file register à zéro avec effet d'état défini par le cœur.	Q: What is CLRF? A: Clear a file register to zero and affect status as defined by the core.
117	Q: Pourquoi CLRF est-il important en programmation PIC Assembly ? R: Mettre un file register à zéro avec effet d'état défini par le cœur. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is CLRF important in PIC Assembly programming? A: Clear a file register to zero and affect status as defined by the core. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
118	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec CLRF ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for CLRF? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
119	Q: Comment valider CLRF sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate CLRF on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
120	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de CLRF. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving CLRF. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
121	Q: Qu'est-ce que literal constant ? R: Une valeur numérique immédiate fournie directement à une instruction/expression assembleur.	Q: What is literal constant? A: An immediate numeric value encoded or supplied directly to an instruction/assembler expression.
122	Q: Pourquoi literal constant est-il important en programmation PIC Assembly ? R: Une valeur numérique immédiate fournie directement à une instruction/expression assembleur. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is literal constant important in PIC Assembly programming? A: An immediate numeric value encoded or supplied directly to an instruction/assembler expression. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
123	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec literal constant ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for literal constant? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
124	Q: Comment valider literal constant sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate literal constant on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
125	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de literal constant. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving literal constant. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.

6. Bit-Oriented Instructions / Instructions bit

#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
126	Q: Qu'est-ce que BSF ? R: Mettre à 1 un bit sélectionné d'un file register.	Q: What is BSF? A: Set a selected bit in a file register.
127	Q: Pourquoi BSF est-il important en programmation PIC Assembly ? R: Mettre à 1 un bit sélectionné d'un file register. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is BSF important in PIC Assembly programming? A: Set a selected bit in a file register. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
128	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec BSF ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for BSF? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
129	Q: Comment valider BSF sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate BSF on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
130	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de BSF. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving BSF. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
131	Q: Qu'est-ce que BCF ? R: Mettre à 0 un bit sélectionné d'un file register.	Q: What is BCF? A: Clear a selected bit in a file register.
132	Q: Pourquoi BCF est-il important en programmation PIC Assembly ? R: Mettre à 0 un bit sélectionné d'un file register. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is BCF important in PIC Assembly programming? A: Clear a selected bit in a file register. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
133	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec BCF ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for BCF? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
134	Q: Comment valider BCF sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate BCF on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
135	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de BCF. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving BCF. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
136	Q: Qu'est-ce que BTFSF ? R: Tester un bit et sauter l'instruction suivante s'il vaut 1.	Q: What is BTFSF? A: Test a bit and skip the next instruction if the tested bit is set.
137	Q: Pourquoi BTFSF est-il important en programmation PIC Assembly ? R: Tester un bit et sauter l'instruction suivante s'il vaut 1. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is BTFSF important in PIC Assembly programming? A: Test a bit and skip the next instruction if the tested bit is set. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
138	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec BTFSF ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for BTFSF? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
139	Q: Comment valider BTFSF sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate BTFSF on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
140	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de BTFSF. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving BTFSF. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
141	Q: Qu'est-ce que BTFSF ? R: Tester un bit et sauter l'instruction suivante s'il vaut 0.	Q: What is BTFSF? A: Test a bit and skip the next instruction if the tested bit is clear.
142	Q: Pourquoi BTFSF est-il important en programmation PIC Assembly ? R: Tester un bit et sauter l'instruction suivante s'il vaut 0. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is BTFSF important in PIC Assembly programming? A: Test a bit and skip the next instruction if the tested bit is clear. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
143	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec BTFSF ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for BTFSF? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
144	Q: Comment valider BTFSC sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate BTFSC on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
145	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de BTFSC. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving BTFSC. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
146	Q: Qu'est-ce que read-modify-write concern ? R: Un problème où une opération bit peut lire un état puis réécrire plusieurs bits, exigeant une conception adaptée au device.	Q: What is read-modify-write concern? A: A hardware/software issue where bit operations on certain port/register implementations can read a pin/register state then rewrite multiple bits, requiring device-aware design.
147	Q: Pourquoi read-modify-write concern est-il important en programmation PIC Assembly ? R: Un problème où une opération bit peut lire un état puis réécrire plusieurs bits, exigeant une conception adaptée au device. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is read-modify-write concern important in PIC Assembly programming? A: A hardware/software issue where bit operations on certain port/register implementations can read a pin/register state then rewrite multiple bits, requiring device-aware design. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
148	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec read-modify-write concern ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for read-modify-write concern? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
149	Q: Comment valider read-modify-write concern sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate read-modify-write concern on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
150	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de read-modify-write concern. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving read-modify-write concern. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.

7. Arithmetic, Logic & Rotate / Arithmétique et logique

#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
151	Q: Qu'est-ce que ADDWF ? R: Additionner w/wreg à un file register avec destination et flags définis.	Q: What is ADDWF? A: Add w/wreg to a file register with destination and flags defined by the core.
152	Q: Pourquoi ADDWF est-il important en programmation PIC Assembly ? R: Additionner w/wreg à un file register avec destination et flags définis. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is ADDWF important in PIC Assembly programming? A: Add w/wreg to a file register with destination and flags defined by the core. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
153	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec ADDWF ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for ADDWF? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
154	Q: Comment valider ADDWF sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate ADDWF on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
155	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de ADDWF. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving ADDWF. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
156	Q: Qu'est-ce que SUBWF ? R: Soustraire w d'un opérande file register selon la sémantique pic.	Q: What is SUBWF? A: Subtract w from a file-register operand according to pic instruction semantics.
157	Q: Pourquoi SUBWF est-il important en programmation PIC Assembly ? R: Soustraire w d'un opérande file register selon la sémantique pic. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is SUBWF important in PIC Assembly programming? A: Subtract w from a file-register operand according to pic instruction semantics. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
158	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec SUBWF ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for SUBWF? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
159	Q: Comment valider SUBWF sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate SUBWF on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
160	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de SUBWF. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving SUBWF. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
161	Q: Qu'est-ce que AND/OR/XOR operations ? R: La logique bit à bit utilisée pour masques, extraction de champs, motifs et protocoles.	Q: What is AND/OR/XOR operations? A: Bitwise logic used for masks, field extraction, setting/toggling patterns, and protocol processing.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
162	Q: Pourquoi AND/OR/XOR operations est-il important en programmation PIC Assembly ? R: La logique bit à bit utilisée pour masques, extraction de champs, motifs et protocoles. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is AND/OR/XOR operations important in PIC Assembly programming? A: Bitwise logic used for masks, field extraction, setting/toggling patterns, and protocol processing. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
163	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec AND/OR/XOR operations ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for AND/OR/XOR operations? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
164	Q: Comment valider AND/OR/XOR operations sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate AND/OR/XOR operations on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
165	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de AND/OR/XOR operations. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving AND/OR/XOR operations. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
166	Q: Qu'est-ce que rotate instruction ? R: Une rotation de bits, souvent via carry, utile pour décalages, algorithmes série et arithmétique multi-octets.	Q: What is rotate instruction? A: A bit rotation operation, often through carry on classic instructions, useful for shifts, serial algorithms, and multi-byte arithmetic.
167	Q: Pourquoi rotate instruction est-il important en programmation PIC Assembly ? R: Une rotation de bits, souvent via carry, utile pour décalages, algorithmes série et arithmétique multi-octets. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is rotate instruction important in PIC Assembly programming? A: A bit rotation operation, often through carry on classic instructions, useful for shifts, serial algorithms, and multi-byte arithmetic. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
168	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec rotate instruction ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for rotate instruction? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
169	Q: Comment valider rotate instruction sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate rotate instruction on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
170	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de rotate instruction. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving rotate instruction. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
171	Q: Qu'est-ce que multi-byte arithmetic ? R: L'arithmétique logique dépassant 8 bits en propageant carry/borrow entre octets.	Q: What is multi-byte arithmetic? A: Software arithmetic extending beyond the native 8-bit data width by propagating carry/borrow across bytes.
172	Q: Pourquoi multi-byte arithmetic est-il important en programmation PIC Assembly ? R: L'arithmétique logique dépassant 8 bits en propageant carry/borrow entre octets. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is multi-byte arithmetic important in PIC Assembly programming? A: Software arithmetic extending beyond the native 8-bit data width by propagating carry/borrow across bytes. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
173	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec multi-byte arithmetic ? R: Pour une addition 16 bits sur cœur 8 bits, additionner d'abord les octets faibles, conserver Carry, puis les octets hauts avec Carry. Tester 0x00FF+0x0001 et les limites.	Q: What calculation or practical check should you know for multi-byte arithmetic? A: For 16-bit addition on an 8-bit core, add low bytes first, preserve carry, then add high bytes plus carry. Test 0x00FF+0x0001 and overflow boundaries.
174	Q: Comment valider multi-byte arithmetic sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate multi-byte arithmetic on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
175	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de multi-byte arithmetic. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving multi-byte arithmetic. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.

8. Branches, CALL & Stack / Branches et pile

#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
176	Q: Qu'est-ce que GOTO ? R: Un transfert de contrôle inconditionnel vers une adresse cible selon les règles architecturales.	Q: What is GOTO? A: An unconditional program-control transfer to a target address within the architectural addressing rules.
177	Q: Pourquoi GOTO est-il important en programmation PIC Assembly ? R: Un transfert de contrôle inconditionnel vers une adresse cible selon les règles architecturales. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is GOTO important in PIC Assembly programming? A: An unconditional program-control transfer to a target address within the architectural addressing rules. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
178	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec GOTO ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for GOTO? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
179	Q: Comment valider GOTO sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate GOTO on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
180	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de GOTO. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving GOTO. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
181	Q: Qu'est-ce que CALL ? R: Un appel de sous-programme sauvegardant une adresse de retour sur la pile matérielle.	Q: What is CALL? A: A subroutine call that saves a return address on the hardware return stack.
182	Q: Pourquoi CALL est-il important en programmation PIC Assembly ? R: Un appel de sous-programme sauvegardant une adresse de retour sur la pile matérielle. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is CALL important in PIC Assembly programming? A: A subroutine call that saves a return address on the hardware return stack. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
183	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec CALL ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for CALL? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
184	Q: Comment valider CALL sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate CALL on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
185	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de CALL. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving CALL. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
186	Q: Qu'est-ce que RETURN ? R: Une instruction de retour restaurant l'exécution depuis la pile.	Q: What is RETURN? A: A subroutine-return instruction restoring execution from the return stack.
187	Q: Pourquoi RETURN est-il important en programmation PIC Assembly ? R: Une instruction de retour restaurant l'exécution depuis la pile. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is RETURN important in PIC Assembly programming? A: A subroutine-return instruction restoring execution from the return stack. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
188	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec RETURN ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for RETURN? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
189	Q: Comment valider RETURN sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate RETURN on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
190	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de RETURN. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving RETURN. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
191	Q: Qu'est-ce que RETFIE ? R: Une instruction de retour d'interruption restaurant le flux et l'état d'interruption selon le cœur.	Q: What is RETFIE? A: Return-from-interrupt instruction that restores program flow and interrupt state according to the core.
192	Q: Pourquoi RETFIE est-il important en programmation PIC Assembly ? R: Une instruction de retour d'interruption restaurant le flux et l'état d'interruption selon le cœur. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is RETFIE important in PIC Assembly programming? A: Return-from-interrupt instruction that restores program flow and interrupt state according to the core. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
193	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec RETFIE ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for RETFIE? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
194	Q: Comment valider RETFIE sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate RETFIE on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
195	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de RETFIE. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving RETFIE. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
196	Q: Qu'est-ce que hardware return stack ? R: Un stockage dédié aux adresses de retour, dont profondeur et accessibilité varient selon le cœur/device.	Q: What is hardware return stack? A: Dedicated storage for return addresses, with depth and accessibility varying by pic core/device.
197	Q: Pourquoi hardware return stack est-il important en programmation PIC Assembly ? R: Un stockage dédié aux adresses de retour, dont profondeur et accessibilité varient selon le cœur/device. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is hardware return stack important in PIC Assembly programming? A: Dedicated storage for return addresses, with depth and accessibility varying by pic core/device. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
198	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec hardware return stack ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for hardware return stack? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
199	Q: Comment valider hardware return stack sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate hardware return stack on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
200	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de hardware return stack. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving hardware return stack. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.

9. Timing, Delays & Cycle Counting / Timing et délais

#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
201	Q: Qu'est-ce que instruction cycle ? R: L'unité de timing cpu; sur pic 8 bits classiques un cycle d'instruction correspond à quatre cycles oscilateur.	Q: What is instruction cycle? A: The cpu timing unit; on classic 8-bit pics one instruction cycle corresponds to four oscillator cycles.
202	Q: Pourquoi instruction cycle est-il important en programmation PIC Assembly ? R: L'unité de timing cpu; sur pic 8 bits classiques un cycle d'instruction correspond à quatre cycles oscilateur. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is instruction cycle important in PIC Assembly programming? A: The cpu timing unit; on classic 8-bit pics one instruction cycle corresponds to four oscillator cycles. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
203	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec instruction cycle ? R: À Fosc=8 MHz, Fcy=2 MHz et Tcy=0,5 µs sur les PIC 8 bits classiques. Une branche de deux cycles prend alors environ 1 µs.	Q: What calculation or practical check should you know for instruction cycle? A: At Fosc=8 MHz, Fcy=2 MHz and Tcy=0.5 µs on classic 8-bit PIC cores. A two-cycle branch then takes about 1 µs.
204	Q: Comment valider instruction cycle sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate instruction cycle on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
205	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de instruction cycle. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving instruction cycle. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
206	Q: Qu'est-ce que branch penalty ? R: Le coût supplémentaire lorsqu'un changement de flux exige de recharger le pipeline.	Q: What is branch penalty? A: Extra cycle cost caused when control flow changes and the instruction pipeline must be refilled.
207	Q: Pourquoi branch penalty est-il important en programmation PIC Assembly ? R: Le coût supplémentaire lorsqu'un changement de flux exige de recharger le pipeline. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is branch penalty important in PIC Assembly programming? A: Extra cycle cost caused when control flow changes and the instruction pipeline must be refilled. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
208	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec branch penalty ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for branch penalty? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
209	Q: Comment valider branch penalty sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate branch penalty on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
210	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de branch penalty. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving branch penalty. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
211	Q: Qu'est-ce que software delay loop ? R: Une boucle déterministe dont le délai total est estimé par comptage des cycles.	Q: What is software delay loop? A: A deterministic instruction loop whose total delay is estimated by counting execution cycles.
212	Q: Pourquoi software delay loop est-il important en programmation PIC Assembly ? R: Une boucle déterministe dont le délai total est estimé par comptage des cycles. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is software delay loop important in PIC Assembly programming? A: A deterministic instruction loop whose total delay is estimated by counting execution cycles. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
213	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec software delay loop ? R: Compter chaque chemin exécuté, y compris le comportement différent de la dernière itération. Multiplier les cycles par Tcy et vérifier à l'oscilloscope si le timing est critique.	Q: What calculation or practical check should you know for software delay loop? A: Count every executed path, including the different final-loop branch/skip behavior. Multiply total cycles by Tcy; verify with a scope when timing matters.
214	Q: Comment valider software delay loop sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate software delay loop on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
215	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de software delay loop. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving software delay loop. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
216	Q: Qu'est-ce que NOP ? R: Une instruction sans opération utilisée pour alignement, petits délais, ajustement timing ou debug.	Q: What is NOP? A: No-operation instruction used for alignment, short delays, timing adjustment, or debug placeholders.
217	Q: Pourquoi NOP est-il important en programmation PIC Assembly ? R: Une instruction sans opération utilisée pour alignement, petits délais, ajustement timing ou debug. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is NOP important in PIC Assembly programming? A: No-operation instruction used for alignment, short delays, timing adjustment, or debug placeholders. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
218	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec NOP ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for NOP? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
219	Q: Comment valider NOP sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate NOP on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
220	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de NOP. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving NOP. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
221	Q: Qu'est-ce que cycle-accurate code ? R: Du code dont le fonctionnement dépend d'un timing d'instructions et chemins précisément connus.	Q: What is cycle-accurate code? A: Code whose correctness depends on precisely known instruction timing and control-flow paths.
222	Q: Pourquoi cycle-accurate code est-il important en programmation PIC Assembly ? R: Du code dont le fonctionnement dépend d'un timing d'instructions et chemins précisément connus. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is cycle-accurate code important in PIC Assembly programming? A: Code whose correctness depends on precisely known instruction timing and control-flow paths. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
223	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec cycle-accurate code ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for cycle-accurate code? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
224	Q: Comment valider cycle-accurate code sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate cycle-accurate code on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
225	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de cycle-accurate code. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving cycle-accurate code. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.

10. GPIO & Digital I/O / GPIO

#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
226	Q: Qu'est-ce que TRIS register ? R: Une famille de registres de direction configurant les broches en entrée ou sortie.	Q: What is TRIS register? A: Direction control register family used to configure pins as inputs or outputs on many pic devices.
227	Q: Pourquoi TRIS register est-il important en programmation PIC Assembly ? R: Une famille de registres de direction configurant les broches en entrée ou sortie. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is TRIS register important in PIC Assembly programming? A: Direction control register family used to configure pins as inputs or outputs on many pic devices. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
228	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec TRIS register ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for TRIS register? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
229	Q: Comment valider TRIS register sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate TRIS register on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
230	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de TRIS register. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving TRIS register. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
231	Q: Qu'est-ce que PORT register ? R: Un registre reflétant ou contrôlant l'état des broches selon le device.	Q: What is PORT register? A: Register reflecting or controlling port pin states according to device behavior.
232	Q: Pourquoi PORT register est-il important en programmation PIC Assembly ? R: Un registre reflétant ou contrôlant l'état des broches selon le device. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is PORT register important in PIC Assembly programming? A: Register reflecting or controlling port pin states according to device behavior. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
233	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec PORT register ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for PORT register? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
234	Q: Comment valider PORT register sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate PORT register on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
235	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de PORT register. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving PORT register. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
236	Q: Qu'est-ce que LAT register ? R: Un registre latch de sortie disponible sur de nombreux devices, aidant à séparer écritures de sortie et lectures de broches.	Q: What is LAT register? A: Output latch register available on many devices, especially pic18/enhanced parts, helping separate output writes from pin reads.
237	Q: Pourquoi LAT register est-il important en programmation PIC Assembly ? R: Un registre latch de sortie disponible sur de nombreux devices, aidant à séparer écritures de sortie et lectures de broches. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is LAT register important in PIC Assembly programming? A: Output latch register available on many devices, especially pic18/enhanced parts, helping separate output writes from pin reads. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
238	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec LAT register ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for LAT register? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
239	Q: Comment valider LAT register sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate LAT register on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
240	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de LAT register. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving LAT register. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
241	Q: Qu'est-ce que analog/digital pin configuration ? R: La configuration nécessaire pour qu'une broche multiplexée utilisée en numérique ne reste pas en mode analogique.	Q: What is analog/digital pin configuration? A: Configuration required on multiplexed pins so a gpio intended for digital use is not left in analog mode.
242	Q: Pourquoi analog/digital pin configuration est-il important en programmation PIC Assembly ? R: La configuration nécessaire pour qu'une broche multiplexée utilisée en numérique ne reste pas en mode analogique. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is analog/digital pin configuration important in PIC Assembly programming? A: Configuration required on multiplexed pins so a gpio intended for digital use is not left in analog mode. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
243	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec analog/digital pin configuration ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for analog/digital pin configuration? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
244	Q: Comment valider analog/digital pin configuration sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate analog/digital pin configuration on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
245	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de analog/digital pin configuration. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving analog/digital pin configuration. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
246	Q: Qu'est-ce que button debounce ? R: Le filtrage matériel ou logiciel des rebonds d'un contact mécanique.	Q: What is button debounce? A: Hardware or software filtering of mechanical switch transitions to avoid multiple false state changes.
247	Q: Pourquoi button debounce est-il important en programmation PIC Assembly ? R: Le filtrage matériel ou logiciel des rebonds d'un contact mécanique. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is button debounce important in PIC Assembly programming? A: Hardware or software filtering of mechanical switch transitions to avoid multiple false state changes. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
248	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec button debounce ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for button debounce? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
249	Q: Comment valider button debounce sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate button debounce on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
250	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de button debounce. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving button debounce. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.

11. Interrupts / Interruptions

#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
251	Q: Qu'est-ce que interrupt vector ? R: L'adresse/mécanisme vectoriel vers lequel l'exécution est transférée lors d'une interruption acceptée.	Q: What is interrupt vector? A: The program-memory address or vector mechanism to which execution transfers on an accepted interrupt.
252	Q: Pourquoi interrupt vector est-il important en programmation PIC Assembly ? R: L'adresse/mécanisme vectoriel vers lequel l'exécution est transférée lors d'une interruption acceptée. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is interrupt vector important in PIC Assembly programming? A: The program-memory address or vector mechanism to which execution transfers on an accepted interrupt. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
253	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec interrupt vector ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for interrupt vector? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
254	Q: Comment valider interrupt vector sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate interrupt vector on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
255	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de interrupt vector. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving interrupt vector. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
256	Q: Qu'est-ce que interrupt flag ? R: Un bit indiquant qu'une condition d'interruption s'est produite.	Q: What is interrupt flag? A: A peripheral/cpu status bit indicating that an interrupt condition occurred.
257	Q: Pourquoi interrupt flag est-il important en programmation PIC Assembly ? R: Un bit indiquant qu'une condition d'interruption s'est produite. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is interrupt flag important in PIC Assembly programming? A: A peripheral/cpu status bit indicating that an interrupt condition occurred. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
258	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec interrupt flag ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for interrupt flag? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
259	Q: Comment valider interrupt flag sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate interrupt flag on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
260	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de interrupt flag. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving interrupt flag. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
261	Q: Qu'est-ce que interrupt enable ? R: Un bit/masque autorisant une source à demander le service cpu.	Q: What is interrupt enable? A: A control bit/mask permitting a particular interrupt source to request cpu service.
262	Q: Pourquoi interrupt enable est-il important en programmation PIC Assembly ? R: Un bit/masque autorisant une source à demander le service cpu. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is interrupt enable important in PIC Assembly programming? A: A control bit/mask permitting a particular interrupt source to request cpu service. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
263	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec interrupt enable ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for interrupt enable? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
264	Q: Comment valider interrupt enable sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate interrupt enable on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
265	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de interrupt enable. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving interrupt enable. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
266	Q: Qu'est-ce que ISR context ? R: L'état cpu/registres devant être préservé ou géré pour reprendre correctement le code interrompu.	Q: What is ISR context? A: Cpu/register state that must be preserved or managed so interrupted code resumes correctly.
267	Q: Pourquoi ISR context est-il important en programmation PIC Assembly ? R: L'état cpu/registres devant être préservé ou géré pour reprendre correctement le code interrompu. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is ISR context important in PIC Assembly programming? A: Cpu/register state that must be preserved or managed so interrupted code resumes correctly. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
268	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec ISR context ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for ISR context? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
269	Q: Comment valider ISR context sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate ISR context on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
270	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de ISR context. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving ISR context. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
271	Q: Qu'est-ce que interrupt latency ? R: Le temps entre l'événement d'interruption et l'exécution du code de service.	Q: What is interrupt latency? A: Time from an interrupt event/request to execution of the relevant service code.
272	Q: Pourquoi interrupt latency est-il important en programmation PIC Assembly ? R: Le temps entre l'événement d'interruption et l'exécution du code de service. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is interrupt latency important in PIC Assembly programming? A: Time from an interrupt event/request to execution of the relevant service code. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
273	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec interrupt latency ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for interrupt latency? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
274	Q: Comment valider interrupt latency sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate interrupt latency on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
275	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de interrupt latency. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving interrupt latency. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.

12. Timers, Counters & Watchdog / Timers et watchdog

#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
276	Q: Qu'est-ce que Timer0 ? R: Un périphérique timer/counter courant dont largeur, horloges, prescaler et fonctions varient selon le device.	Q: What is Timer0? A: A commonly available timer/counter peripheral whose exact width, clock sources, prescaler, and features vary by device.
277	Q: Pourquoi Timer0 est-il important en programmation PIC Assembly ? R: Un périphérique timer/counter courant dont largeur, horloges, prescaler et fonctions varient selon le device. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is Timer0 important in PIC Assembly programming? A: A commonly available timer/counter peripheral whose exact width, clock sources, prescaler, and features vary by device. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
278	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec Timer0 ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for Timer0? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
279	Q: Comment valider Timer0 sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate Timer0 on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
280	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de Timer0. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving Timer0. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
281	Q: Qu'est-ce que Timer1 ? R: Un timer courant avec largeur/fonctions dépendant du device et choix d'horloge interne/externe.	Q: What is Timer1? A: A commonly available timer with device-dependent width/features and selectable internal/external clocking on many pics.
282	Q: Pourquoi Timer1 est-il important en programmation PIC Assembly ? R: Un timer courant avec largeur/fonctions dépendant du device et choix d'horloge interne/externe. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is Timer1 important in PIC Assembly programming? A: A commonly available timer with device-dependent width/features and selectable internal/external clocking on many pics. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
283	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec Timer1 ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for Timer1? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
284	Q: Comment valider Timer1 sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate Timer1 on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
285	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de Timer1. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving Timer1. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
286	Q: Qu'est-ce que prescaler ? R: Un diviseur réduisant la fréquence avant incrément du timer.	Q: What is prescaler? A: A divider reducing an input clock rate before it increments a timer/counter.
287	Q: Pourquoi prescaler est-il important en programmation PIC Assembly ? R: Un diviseur réduisant la fréquence avant incrément du timer. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is prescaler important in PIC Assembly programming? A: A divider reducing an input clock rate before it increments a timer/counter. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
288	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec prescaler ? R: Si l'horloge timer vaut 1 MHz et le prescaler 1:8, le tick vaut 8 µs. Vérifier d'abord l'horloge propre au timer/device.	Q: What calculation or practical check should you know for prescaler? A: If a timer clock is 1 MHz and prescaler is 1:8, the timer tick is 8 µs. Device-specific timer clocking must be verified first.
289	Q: Comment valider prescaler sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate prescaler on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
290	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de prescaler. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving prescaler. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
291	Q: Qu'est-ce que timer preload ? R: Une valeur initiale choisie pour obtenir overflow/match après un nombre souhaité de ticks.	Q: What is timer preload? A: An initial counter value chosen so overflow/match occurs after a desired number of ticks.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
292	Q: Pourquoi timer preload est-il important en programmation PIC Assembly ? R: Une valeur initiale choisie pour obtenir overflow/match après un nombre souhaité de ticks. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is timer preload important in PIC Assembly programming? A: An initial counter value chosen so overflow/match occurs after a desired number of ticks. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
293	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec timer preload ? R: Pour un timer 16 bits, N counts souhaités donnent preload=65536-N pour un overflow simple. Inclure prescaler et budget de latence ISR.	Q: What calculation or practical check should you know for timer preload? A: For a 16-bit timer, desired counts N imply preload=65536-N when simple overflow timing is used. Include prescaler and ISR latency/error budget.
294	Q: Comment valider timer preload sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate timer preload on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
295	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de timer preload. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving timer preload. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
296	Q: Qu'est-ce que watchdog timer ? R: Un mécanisme de timeout destiné à réinitialiser/récupérer le système si le logiciel cesse de le servir.	Q: What is watchdog timer? A: An independent or semi-independent timeout mechanism intended to reset or recover a system when software stops servicing it.
297	Q: Pourquoi watchdog timer est-il important en programmation PIC Assembly ? R: Un mécanisme de timeout destiné à réinitialiser/récupérer le système si le logiciel cesse de le servir. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is watchdog timer important in PIC Assembly programming? A: An independent or semi-independent timeout mechanism intended to reset or recover a system when software stops servicing it. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
298	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec watchdog timer ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for watchdog timer? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
299	Q: Comment valider watchdog timer sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate watchdog timer on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
300	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de watchdog timer. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving watchdog timer. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.

13. ADC & Analog Measurements / ADC

#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
301	Q: Qu'est-ce que ADC resolution ? R: Le nombre de bits numériques représentant la plage de conversion analogique.	Q: What is ADC resolution? A: Number of digital bits used to represent the analog conversion range.
302	Q: Pourquoi ADC resolution est-il important en programmation PIC Assembly ? R: Le nombre de bits numériques représentant la plage de conversion analogique. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is ADC resolution important in PIC Assembly programming? A: Number of digital bits used to represent the analog conversion range. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
303	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec ADC resolution ? R: Pour un ADC idéal 10 bits sur 0-5 V, un pas vaut environ $5/1023 \approx 4.89$ mV avec le code pleine échelle 1023.	Q: What calculation or practical check should you know for ADC resolution? A: For an ideal 10-bit ADC over 0-5 V, one code step is about $5/1023 \approx 4.89$ mV when using full-scale code 1023.
304	Q: Comment valider ADC resolution sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate ADC resolution on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
305	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de ADC resolution. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving ADC resolution. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
306	Q: Qu'est-ce que Vref ? R: La ou les tensions de référence définissant l'échelle de conversion.	Q: What is Vref? A: The adc reference voltage(s) defining the conversion scale, subject to device configuration.
307	Q: Pourquoi Vref est-il important en programmation PIC Assembly ? R: La ou les tensions de référence définissant l'échelle de conversion. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is Vref important in PIC Assembly programming? A: The adc reference voltage(s) defining the conversion scale, subject to device configuration. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
308	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec Vref ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for Vref? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
309	Q: Comment valider Vref sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate Vref on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
310	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de Vref. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving Vref. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
311	Q: Qu'est-ce que acquisition time ? R: Le temps laissé au sample/hold pour se charger suffisamment depuis la source.	Q: What is acquisition time? A: Time allowed for the adc sample/hold network to charge sufficiently from the selected source.
312	Q: Pourquoi acquisition time est-il important en programmation PIC Assembly ? R: Le temps laissé au sample/hold pour se charger suffisamment depuis la source. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is acquisition time important in PIC Assembly programming? A: Time allowed for the adc sample/hold network to charge sufficiently from the selected source. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
313	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec acquisition time ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for acquisition time? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
314	Q: Comment valider acquisition time sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate acquisition time on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
315	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de acquisition time. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving acquisition time. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
316	Q: Qu'est-ce que ADC result alignment ? R: La justification gauche/droite des bits de résultat dans les registres.	Q: What is ADC result alignment? A: Left/right justification of conversion-result bits in result registers, depending on device configuration.
317	Q: Pourquoi ADC result alignment est-il important en programmation PIC Assembly ? R: La justification gauche/droite des bits de résultat dans les registres. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is ADC result alignment important in PIC Assembly programming? A: Left/right justification of conversion-result bits in result registers, depending on device configuration. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
318	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec ADC result alignment ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for ADC result alignment? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
319	Q: Comment valider ADC result alignment sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate ADC result alignment on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
320	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de ADC result alignment. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving ADC result alignment. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
321	Q: Qu'est-ce que ADC source impedance ? R: L'impédance source interagissant avec le condensateur d'échantillonnage et le temps d'acquisition.	Q: What is ADC source impedance? A: Source resistance/impedance that interacts with the adc sampling capacitor and acquisition-time requirement.
322	Q: Pourquoi ADC source impedance est-il important en programmation PIC Assembly ? R: L'impédance source interagissant avec le condensateur d'échantillonnage et le temps d'acquisition. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is ADC source impedance important in PIC Assembly programming? A: Source resistance/impedance that interacts with the adc sampling capacitor and acquisition-time requirement. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
323	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec ADC source impedance ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for ADC source impedance? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
324	Q: Comment valider ADC source impedance sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate ADC source impedance on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
325	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de ADC source impedance. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving ADC source impedance. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.

14. CCP/PWM & Output Control / PWM

#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
326	Q: Qu'est-ce que CCP module ? R: Une famille de périphériques capture/compare/pwm pour mesure timing, comparaison et génération pwm.	Q: What is CCP module? A: Capture/compare/pwm peripheral family used for timing measurement, event comparison, and pwm generation on many pics.
327	Q: Pourquoi CCP module est-il important en programmation PIC Assembly ? R: Une famille de périphériques capture/compare/pwm pour mesure timing, comparaison et génération pwm. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is CCP module important in PIC Assembly programming? A: Capture/compare/pwm peripheral family used for timing measurement, event comparison, and pwm generation on many pics. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
328	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec CCP module ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for CCP module? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
329	Q: Comment valider CCP module sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate CCP module on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
330	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de CCP module. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving CCP module. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
331	Q: Qu'est-ce que PWM frequency ? R: La fréquence de répétition pwm déterminée par horloge, période timer et prescaler selon le device.	Q: What is PWM frequency? A: Repetition rate of a pwm waveform determined by peripheral clock, timer period, and prescaling according to the device.
332	Q: Pourquoi PWM frequency est-il important en programmation PIC Assembly ? R: La fréquence de répétition pwm déterminée par horloge, période timer et prescaler selon le device. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is PWM frequency important in PIC Assembly programming? A: Repetition rate of a pwm waveform determined by peripheral clock, timer period, and prescaling according to the device. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
333	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec PWM frequency ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for PWM frequency? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
334	Q: Comment valider PWM frequency sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate PWM frequency on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
335	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de PWM frequency. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving PWM frequency. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
336	Q: Qu'est-ce que PWM duty cycle ? R: La fraction de la période pwm pendant laquelle la sortie est active.	Q: What is PWM duty cycle? A: Fraction of each pwm period for which the output is active.
337	Q: Pourquoi PWM duty cycle est-il important en programmation PIC Assembly ? R: La fraction de la période pwm pendant laquelle la sortie est active. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is PWM duty cycle important in PIC Assembly programming? A: Fraction of each pwm period for which the output is active. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
338	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec PWM duty cycle ? R: Duty=Ton/Tperiodx100 %. Exemple : 0,25 ms haut sur une période de 1 ms donne 25 %. Les formules registres dépendent du device.	Q: What calculation or practical check should you know for PWM duty cycle? A: Duty=Ton/Tperiodx100%. Example: 0.25 ms high in a 1 ms period gives 25%. Register formulas are device-specific.
339	Q: Comment valider PWM duty cycle sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate PWM duty cycle on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
340	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de PWM duty cycle. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving PWM duty cycle. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
341	Q: Qu'est-ce que capture mode ? R: La capture matérielle d'une valeur timer lors d'un événement d'entrée.	Q: What is capture mode? A: Hardware capture of a timer value on an input event for precise timing measurement.
342	Q: Pourquoi capture mode est-il important en programmation PIC Assembly ? R: La capture matérielle d'une valeur timer lors d'un événement d'entrée. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is capture mode important in PIC Assembly programming? A: Hardware capture of a timer value on an input event for precise timing measurement. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
343	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec capture mode ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for capture mode? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
344	Q: Comment valider capture mode sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate capture mode on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
345	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de capture mode. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving capture mode. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
346	Q: Qu'est-ce que compare mode ? R: La comparaison matérielle d'un timer à une valeur programmée pour déclencher une action.	Q: What is compare mode? A: Hardware comparison of a timer against a programmed value to trigger an event/output/action.
347	Q: Pourquoi compare mode est-il important en programmation PIC Assembly ? R: La comparaison matérielle d'un timer à une valeur programmée pour déclencher une action. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is compare mode important in PIC Assembly programming? A: Hardware comparison of a timer against a programmed value to trigger an event/output/action. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
348	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec compare mode ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for compare mode? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
349	Q: Comment valider compare mode sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate compare mode on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
350	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de compare mode. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving compare mode. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.

15. UART / Serial Communication

#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
351	Q: Qu'est-ce que UART baud rate ? R: Le timing série configuré à partir d'une horloge périphérique et du générateur de baud.	Q: What is UART baud rate? A: Serial symbol/bit timing configured from a peripheral clock and baud-rate generator settings.
352	Q: Pourquoi UART baud rate est-il important en programmation PIC Assembly ? R: Le timing série configuré à partir d'une horloge périphérique et du générateur de baud. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is UART baud rate important in PIC Assembly programming? A: Serial symbol/bit timing configured from a peripheral clock and baud-rate generator settings. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
353	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec UART baud rate ? R: Partir de l'équation du générateur de baud du device, calculer le registre, puis le baud réel et l'erreur en %. Vérifier la tolérance du correspondant.	Q: What calculation or practical check should you know for UART baud rate? A: Start from the device's baud-generator equation, calculate the register value, then compute actual baud and percent error. Validate against the peer tolerance.
354	Q: Comment valider UART baud rate sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate UART baud rate on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
355	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de UART baud rate. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving UART baud rate. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
356	Q: Qu'est-ce que TXREG/transmit register concept ? R: Le registre de données d'émission alimentant le matériel uart sur de nombreux pic.	Q: What is TXREG/transmit register concept? A: The software-facing transmit data register feeding the uart transmit hardware on many pic devices.
357	Q: Pourquoi TXREG/transmit register concept est-il important en programmation PIC Assembly ? R: Le registre de données d'émission alimentant le matériel uart sur de nombreux pic. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is TXREG/transmit register concept important in PIC Assembly programming? A: The software-facing transmit data register feeding the uart transmit hardware on many pic devices. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
358	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec TXREG/transmit register concept ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for TXREG/transmit register concept? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
359	Q: Comment valider TXREG/transmit register concept sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate TXREG/transmit register concept on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
360	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de TXREG/transmit register concept. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving TXREG/transmit register concept. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
361	Q: Qu'est-ce que receive register concept ? R: Le registre/buffer depuis lequel le logiciel lit les données uart reçues.	Q: What is receive register concept? A: The receive data register/buffer from which software reads received uart data.
362	Q: Pourquoi receive register concept est-il important en programmation PIC Assembly ? R: Le registre/buffer depuis lequel le logiciel lit les données uart reçues. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is receive register concept important in PIC Assembly programming? A: The receive data register/buffer from which software reads received uart data. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
363	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec receive register concept ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for receive register concept? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
364	Q: Comment valider receive register concept sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate receive register concept on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
365	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de receive register concept. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving receive register concept. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
366	Q: Qu'est-ce que framing error ? R: Une erreur indiquant que le stop bit attendu n'a pas été observé.	Q: What is framing error? A: A receive error indicating the expected stop-bit framing was not observed.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
367	Q: Pourquoi framing error est-il important en programmation PIC Assembly ? R: Une erreur indiquant que le stop bit attendu n'a pas été observé. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is framing error important in PIC Assembly programming? A: A receive error indicating the expected stop-bit framing was not observed. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
368	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec framing error ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for framing error? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
369	Q: Comment valider framing error sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate framing error on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
370	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de framing error. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving framing error. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
371	Q: Qu'est-ce que overrun error ? R: Une condition où de nouvelles données arrivent avant traitement du buffer précédent, nécessitant une récupération propre au device.	Q: What is overrun error? A: A receive condition where new data arrives before previous buffered data is serviced, requiring device-specific recovery.
372	Q: Pourquoi overrun error est-il important en programmation PIC Assembly ? R: Une condition où de nouvelles données arrivent avant traitement du buffer précédent, nécessitant une récupération propre au device. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is overrun error important in PIC Assembly programming? A: A receive condition where new data arrives before previous buffered data is serviced, requiring device-specific recovery. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
373	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec overrun error ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for overrun error? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
374	Q: Comment valider overrun error sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate overrun error on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
375	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de overrun error. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving overrun error. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.

16. SPI & I²C / Interfaces série

#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
376	Q: Qu'est-ce que SPI master ? R: Le dispositif contrôlant l'horloge spi et le timing de transaction.	Q: What is SPI master? A: The device controlling spi clock and transaction timing in a master/slave exchange.
377	Q: Pourquoi SPI master est-il important en programmation PIC Assembly ? R: Le dispositif contrôlant l'horloge spi et le timing de transaction. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is SPI master important in PIC Assembly programming? A: The device controlling spi clock and transaction timing in a master/slave exchange. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
378	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec SPI master ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for SPI master? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
379	Q: Comment valider SPI master sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate SPI master on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
380	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de SPI master. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving SPI master. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
381	Q: Qu'est-ce que SPI mode ? R: La relation polarité/phase d'horloge devant correspondre entre dispositifs.	Q: What is SPI mode? A: Clock polarity/phase relationship that must match between communicating devices.
382	Q: Pourquoi SPI mode est-il important en programmation PIC Assembly ? R: La relation polarité/phase d'horloge devant correspondre entre dispositifs. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is SPI mode important in PIC Assembly programming? A: Clock polarity/phase relationship that must match between communicating devices. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
383	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec SPI mode ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for SPI mode? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
384	Q: Comment valider SPI mode sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate SPI mode on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
385	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de SPI mode. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving SPI mode. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
386	Q: Qu'est-ce que I ² C open-drain behavior ? R: Une signalisation où les dispositifs tirent activement les lignes à 0 et utilisent des pull-ups pour l'état haut.	Q: What is I ² C open-drain behavior? A: Bus signaling where devices actively pull lines low and rely on pull-ups for the high state.
387	Q: Pourquoi I ² C open-drain behavior est-il important en programmation PIC Assembly ? R: Une signalisation où les dispositifs tirent activement les lignes à 0 et utilisent des pull-ups pour l'état haut. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is I ² C open-drain behavior important in PIC Assembly programming? A: Bus signaling where devices actively pull lines low and rely on pull-ups for the high state. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
388	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec I ² C open-drain behavior ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for I ² C open-drain behavior? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
389	Q: Comment valider I ² C open-drain behavior sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate I ² C open-drain behavior on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
390	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de I ² C open-drain behavior. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving I ² C open-drain behavior. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
391	Q: Qu'est-ce que I ² C ACK/NACK ? R: Le bit d'acquiescement/non-acquiescement contrôlant le flux octet par octet.	Q: What is I ² C ACK/NACK? A: Receiver acknowledgment or non-acknowledgment bit used to control byte-level transfer flow.
392	Q: Pourquoi I ² C ACK/NACK est-il important en programmation PIC Assembly ? R: Le bit d'acquiescement/non-acquiescement contrôlant le flux octet par octet. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is I ² C ACK/NACK important in PIC Assembly programming? A: Receiver acknowledgment or non-acknowledgment bit used to control byte-level transfer flow. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
393	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec I ² C ACK/NACK ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for I ² C ACK/NACK? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
394	Q: Comment valider I ² C ACK/NACK sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate I ² C ACK/NACK on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
395	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de I ² C ACK/NACK. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving I ² C ACK/NACK. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
396	Q: Qu'est-ce que I ² C pull-up sizing ? R: Le choix de résistance équilibrant temps de montée, capacité bus, courant sink, tension et timing protocole.	Q: What is I ² C pull-up sizing? A: Selection of pull-up resistance balancing rise time, bus capacitance, sink current, voltage, and protocol timing.
397	Q: Pourquoi I ² C pull-up sizing est-il important en programmation PIC Assembly ? R: Le choix de résistance équilibrant temps de montée, capacité bus, courant sink, tension et timing protocole. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is I ² C pull-up sizing important in PIC Assembly programming? A: Selection of pull-up resistance balancing rise time, bus capacitance, sink current, voltage, and protocol timing. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
398	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec I ² C pull-up sizing ? R: Vérifier les deux limites : le temps de montée fixe une résistance maximale via R _p -C _{bus} , tandis que le courant sink/niveau bas fixe une résistance minimale.	Q: What calculation or practical check should you know for I ² C pull-up sizing? A: Check both limits: rise time sets an upper resistance through R _p -C _{bus} , while sink-current/low-level requirements set a lower resistance.
399	Q: Comment valider I ² C pull-up sizing sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate I ² C pull-up sizing on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
400	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de I ² C pull-up sizing. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving I ² C pull-up sizing. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.

17. EEPROM, Flash & Configuration / Mémoire non volatile

#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
401	Q: Qu'est-ce que data EEPROM ? R: Une mémoire de données non volatile pour paramètres/calibration/état, avec endurance d'écriture finie.	Q: What is data EEPROM? A: Nonvolatile data storage available on many pic devices for parameters/calibration/state, with finite write endurance.
402	Q: Pourquoi data EEPROM est-il important en programmation PIC Assembly ? R: Une mémoire de données non volatile pour paramètres/calibration/état, avec endurance d'écriture finie. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is data EEPROM important in PIC Assembly programming? A: Nonvolatile data storage available on many pic devices for parameters/calibration/state, with finite write endurance. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
403	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec data EEPROM ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for data EEPROM? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
404	Q: Comment valider data EEPROM sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate data EEPROM on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
405	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de data EEPROM. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving data EEPROM. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
406	Q: Qu'est-ce que EEPROM write sequence ? R: Une séquence protégée propre au device pour lancer les écritures non volatiles.	Q: What is EEPROM write sequence? A: A protected device-specific register sequence used to initiate nonvolatile writes safely.
407	Q: Pourquoi EEPROM write sequence est-il important en programmation PIC Assembly ? R: Une séquence protégée propre au device pour lancer les écritures non volatiles. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is EEPROM write sequence important in PIC Assembly programming? A: A protected device-specific register sequence used to initiate nonvolatile writes safely. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
408	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec EEPROM write sequence ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for EEPROM write sequence? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
409	Q: Comment valider EEPROM write sequence sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate EEPROM write sequence on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
410	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de EEPROM write sequence. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving EEPROM write sequence. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
411	Q: Qu'est-ce que program Flash write ? R: Une opération d'auto-programmation/programmation de la mémoire programme selon capacités et règles du device.	Q: What is program Flash write? A: Self-programming or programming operation on program memory, available only according to device capabilities and rules.
412	Q: Pourquoi program Flash write est-il important en programmation PIC Assembly ? R: Une opération d'auto-programmation/programmation de la mémoire programme selon capacités et règles du device. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is program Flash write important in PIC Assembly programming? A: Self-programming or programming operation on program memory, available only according to device capabilities and rules. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
413	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec program Flash write ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for program Flash write? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
414	Q: Comment valider program Flash write sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate program Flash write on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
415	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de program Flash write. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving program Flash write. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
416	Q: Qu'est-ce que configuration bits ? R: Des réglages non volatils sélectionnant oscillateur, watchdog, protection, reset et autres comportements.	Q: What is configuration bits? A: Nonvolatile device configuration settings selecting oscillator, watchdog, protection, reset, programming, and other startup behavior.
417	Q: Pourquoi configuration bits est-il important en programmation PIC Assembly ? R: Des réglages non volatils sélectionnant oscillateur, watchdog, protection, reset et autres comportements. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is configuration bits important in PIC Assembly programming? A: Nonvolatile device configuration settings selecting oscillator, watchdog, protection, reset, programming, and other startup behavior. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
418	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec configuration bits ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for configuration bits? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
419	Q: Comment valider configuration bits sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate configuration bits on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
420	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de configuration bits. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving configuration bits. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
421	Q: Qu'est-ce que write endurance ? R: Le nombre fini de cycles effacement/écriture nécessitant une conception tenant compte de l'usure.	Q: What is write endurance? A: Finite number of guaranteed/characterized nonvolatile erase/write cycles requiring wear-aware design.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
422	Q: Pourquoi write endurance est-il important en programmation PIC Assembly ? R: Le nombre fini de cycles effacement/écriture nécessitant une conception tenant compte de l'usure. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is write endurance important in PIC Assembly programming? A: Finite number of guaranteed/characterized nonvolatile erase/write cycles requiring wear-aware design. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
423	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec write endurance ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for write endurance? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
424	Q: Comment valider write endurance sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate write endurance on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
425	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de write endurance. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving write endurance. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.

18. Assembler, Directives & Macros / Assembleur et directives

#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
426	Q: Qu'est-ce que assembler directive ? R: Une commande source contrôlant assemblage, sections, symboles, données, configuration ou linkage plutôt qu'une instruction cpu normale.	Q: What is assembler directive? A: A source command controlling assembly, sections, symbols, data, configuration, or linkage rather than generating a normal cpu instruction.
427	Q: Pourquoi assembler directive est-il important en programmation PIC Assembly ? R: Une commande source contrôlant assemblage, sections, symboles, données, configuration ou linkage plutôt qu'une instruction cpu normale. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is assembler directive important in PIC Assembly programming? A: A source command controlling assembly, sections, symbols, data, configuration, or linkage rather than generating a normal cpu instruction. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
428	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec assembler directive ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for assembler directive? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
429	Q: Comment valider assembler directive sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate assembler directive on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
430	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de assembler directive. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving assembler directive. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
431	Q: Qu'est-ce que label ? R: Un nom symbolique représentant une adresse code ou données.	Q: What is label? A: A symbolic name representing a code or data address.
432	Q: Pourquoi label est-il important en programmation PIC Assembly ? R: Un nom symbolique représentant une adresse code ou données. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is label important in PIC Assembly programming? A: A symbolic name representing a code or data address. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
433	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec label ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for label? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
434	Q: Comment valider label sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate label on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
435	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de label. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving label. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
436	Q: Qu'est-ce que EQU/set-style constant ? R: Une définition de constante symbolique à l'assemblage, dont syntaxe exacte dépend de l'assembleur.	Q: What is EQU/set-style constant? A: An assembler-time symbolic constant definition, with exact directive syntax depending on assembler.
437	Q: Pourquoi EQU/set-style constant est-il important en programmation PIC Assembly ? R: Une définition de constante symbolique à l'assemblage, dont syntaxe exacte dépend de l'assembleur. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is EQU/set-style constant important in PIC Assembly programming? A: An assembler-time symbolic constant definition, with exact directive syntax depending on assembler. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
438	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec EQU/set-style constant ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for EQU/set-style constant? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
439	Q: Comment valider EQU/set-style constant sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate EQU/set-style constant on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
440	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de EQU/set-style constant. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving EQU/set-style constant. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
441	Q: Qu'est-ce que macro ? R: Un mécanisme d'expansion source paramétré pour réutiliser des motifs assembleur.	Q: What is macro? A: A parameterized source-expansion mechanism used to reuse assembly patterns.
442	Q: Pourquoi macro est-il important en programmation PIC Assembly ? R: Un mécanisme d'expansion source paramétré pour réutiliser des motifs assembleur. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is macro important in PIC Assembly programming? A: A parameterized source-expansion mechanism used to reuse assembly patterns. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
443	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec macro ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for macro? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
444	Q: Comment valider macro sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate macro on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
445	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de macro. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving macro. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
446	Q: Qu'est-ce que section/linker placement ? R: L'organisation code/données en sections dont les adresses finales sont déterminées par assembleur/linker.	Q: What is section/linker placement? A: Organization of code/data into sections whose final addresses are determined by assembler/linker rules.
447	Q: Pourquoi section/linker placement est-il important en programmation PIC Assembly ? R: L'organisation code/données en sections dont les adresses finales sont déterminées par assembleur/linker. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is section/linker placement important in PIC Assembly programming? A: Organization of code/data into sections whose final addresses are determined by assembler/linker rules. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
448	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec section/linker placement ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for section/linker placement? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
449	Q: Comment valider section/linker placement sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate section/linker placement on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
450	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de section/linker placement. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving section/linker placement. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.

19. MPLAB X, MPASM & PIC-AS / Outils Microchip

#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
451	Q: Qu'est-ce que MPLAB X IDE ? R: L'environnement de développement microchip utilisé pour projets, build, programmation et debug.	Q: What is MPLAB X IDE? A: Microchip's integrated development environment used for projects, build tools, programming, and debugging.
452	Q: Pourquoi MPLAB X IDE est-il important en programmation PIC Assembly ? R: L'environnement de développement microchip utilisé pour projets, build, programmation et debug. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is MPLAB X IDE important in PIC Assembly programming? A: Microchip's integrated development environment used for projects, build tools, programming, and debugging. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
453	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec MPLAB X IDE ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for MPLAB X IDE? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
454	Q: Comment valider MPLAB X IDE sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate MPLAB X IDE on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
455	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de MPLAB X IDE. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving MPLAB X IDE. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
456	Q: Qu'est-ce que MPASM ? R: L'ancien toolchain assembleur microchip utilisé par de nombreux projets pic et non installé par défaut dans les versions actuelles de mlab x.	Q: What is MPASM? A: Microchip's legacy assembler toolchain used by many older pic projects and no longer installed by default with current mlab x versions.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
457	Q: Pourquoi MPASM est-il important en programmation PIC Assembly ? R: L'ancien toolchain assembleur microchip utilisé par de nombreux projets pic et non installé par défaut dans les versions actuelles de mlab x. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is MPASM important in PIC Assembly programming? A: Microchip's legacy assembler toolchain used by many older pic projects and no longer installed by default with current mlab x versions. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
458	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec MPASM ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for MPASM? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
459	Q: Comment valider MPASM sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate MPASM on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
460	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de MPASM. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving MPASM. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
461	Q: Qu'est-ce que MPLAB XC8 PIC Assembler ? R: L'assembleur pic actuel fourni dans le toolchain xc8 pour pic 8 bits supportés.	Q: What is MPLAB XC8 PIC Assembler? A: The current pic assembler supplied in the xc8 toolchain for supported 8-bit pic devices.
462	Q: Pourquoi MPLAB XC8 PIC Assembler est-il important en programmation PIC Assembly ? R: L'assembleur pic actuel fourni dans le toolchain xc8 pour pic 8 bits supportés. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is MPLAB XC8 PIC Assembler important in PIC Assembly programming? A: The current pic assembler supplied in the xc8 toolchain for supported 8-bit pic devices. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
463	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec MPLAB XC8 PIC Assembler ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for MPLAB XC8 PIC Assembler? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
464	Q: Comment valider MPLAB XC8 PIC Assembler sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate MPLAB XC8 PIC Assembler on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
465	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de MPLAB XC8 PIC Assembler. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving MPLAB XC8 PIC Assembler. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
466	Q: Qu'est-ce que pic-as driver ? R: Le driver ligne de commande utilisé pour assembler et linker les projets pic assembler xc8.	Q: What is pic-as driver? A: The command-line driver used to assemble and link mlab xc8 pic assembler projects.
467	Q: Pourquoi pic-as driver est-il important en programmation PIC Assembly ? R: Le driver ligne de commande utilisé pour assembler et linker les projets pic assembler xc8. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is pic-as driver important in PIC Assembly programming? A: The command-line driver used to assemble and link mlab xc8 pic assembler projects. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
468	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec pic-as driver ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for pic-as driver? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
469	Q: Comment valider pic-as driver sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate pic-as driver on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
470	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de pic-as driver. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving pic-as driver. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
471	Q: Qu'est-ce que MPASM migration ? R: La conversion de projets legacy où directives, expressions, sections, fichiers et build/link peuvent changer même si les séquences d'instructions restent conceptuellement proches.	Q: What is MPASM migration? A: Conversion of legacy projects where directives, expressions, sections, files, and build/link behavior may require changes even when instruction sequences remain conceptually similar.
472	Q: Pourquoi MPASM migration est-il important en programmation PIC Assembly ? R: La conversion de projets legacy où directives, expressions, sections, fichiers et build/link peuvent changer même si les séquences d'instructions restent conceptuellement proches. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is MPASM migration important in PIC Assembly programming? A: Conversion of legacy projects where directives, expressions, sections, files, and build/link behavior may require changes even when instruction sequences remain conceptually similar. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
473	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec MPASM migration ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for MPASM migration? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
474	Q: Comment valider MPASM migration sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate MPASM migration on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
475	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de MPASM migration. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving MPASM migration. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.



20. Bring-Up, Debugging & Senior RCA / Bring-up et RCA

#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
476	Q: Qu'est-ce que first power-up ? R: La première mise sous tension contrôlée avec limitation de courant, vérification alimentation/clock/reset et firmware minimal.	Q: What is first power-up? A: Controlled initial board execution with current limiting, supply/clock/reset checks, and minimal firmware before enabling all peripherals.
477	Q: Pourquoi first power-up est-il important en programmation PIC Assembly ? R: La première mise sous tension contrôlée avec limitation de courant, vérification alimentation/clock/reset et firmware minimal. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is first power-up important in PIC Assembly programming? A: Controlled initial board execution with current limiting, supply/clock/reset checks, and minimal firmware before enabling all peripherals. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
478	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec first power-up ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for first power-up? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
479	Q: Comment valider first power-up sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate first power-up on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
480	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de first power-up. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving first power-up. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
481	Q: Qu'est-ce que ICSP/debug interface ? R: La connexion de programmation/debug in-circuit utilisée pour programmer et inspecter un pic.	Q: What is ICSP/debug interface? A: In-circuit programming/debug connection used to program and inspect a pic device with supported tools.
482	Q: Pourquoi ICSP/debug interface est-il important en programmation PIC Assembly ? R: La connexion de programmation/debug in-circuit utilisée pour programmer et inspecter un pic. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is ICSP/debug interface important in PIC Assembly programming? A: In-circuit programming/debug connection used to program and inspect a pic device with supported tools. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
483	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec ICSP/debug interface ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for ICSP/debug interface? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
484	Q: Comment valider ICSP/debug interface sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate ICSP/debug interface on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
485	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de ICSP/debug interface. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving ICSP/debug interface. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
486	Q: Qu'est-ce que register-level debugging ? R: L'inspection des sfr, flags, mémoire, pile/état de contrôle et timing périphérique pour trouver les défauts.	Q: What is register-level debugging? A: Inspection of sfrs, flags, memory, stack/control state, and peripheral timing to identify low-level faults.
487	Q: Pourquoi register-level debugging est-il important en programmation PIC Assembly ? R: L'inspection des sfr, flags, mémoire, pile/état de contrôle et timing périphérique pour trouver les défauts. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is register-level debugging important in PIC Assembly programming? A: Inspection of sfrs, flags, memory, stack/control state, and peripheral timing to identify low-level faults. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
488	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec register-level debugging ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for register-level debugging? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
489	Q: Comment valider register-level debugging sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate register-level debugging on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
490	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de register-level debugging. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving register-level debugging. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/adresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
491	Q: Qu'est-ce que logic analyzer validation ? R: La mesure du timing/protocole numérique uart, spi, i ² c, pwm, interruptions et gpio.	Q: What is logic analyzer validation? A: Measurement of digital timing/protocol behavior such as uart, spi, i ² c, pwm, interrupts, and gpio sequences.
492	Q: Pourquoi logic analyzer validation est-il important en programmation PIC Assembly ? R: La mesure du timing/protocole numérique uart, spi, i ² c, pwm, interruptions et gpio. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is logic analyzer validation important in PIC Assembly programming? A: Measurement of digital timing/protocol behavior such as uart, spi, i ² c, pwm, interrupts, and gpio sequences. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
493	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec logic analyzer validation ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for logic analyzer validation? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
494	Q: Comment valider logic analyzer validation sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate logic analyzer validation on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.



#	FRANÇAIS — Question / Réponse	ENGLISH — Question / Answer
495	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de logic analyzer validation. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving logic analyzer validation. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.
496	Q: Qu'est-ce que senior PIC RCA ? R: La séparation systématique des défauts hardware, erreurs clock/reset/config, registres, banking/adressage, interruptions/timing, protocoles et différences de toolchain/build.	Q: What is senior PIC RCA? A: Systematic separation of hardware faults, clock/reset/configuration errors, register setup, banking/addressing bugs, interrupt/timing defects, protocol issues, and toolchain/build differences.
497	Q: Pourquoi senior PIC RCA est-il important en programmation PIC Assembly ? R: La séparation systématique des défauts hardware, erreurs clock/reset/config, registres, banking/adressage, interruptions/timing, protocoles et différences de toolchain/build. Il faut comprendre son impact réel sur les registres, la mémoire, le timing et le matériel, pas seulement mémoriser la syntaxe.	Q: Why is senior PIC RCA important in PIC Assembly programming? A: Systematic separation of hardware faults, clock/reset/configuration errors, register setup, banking/addressing bugs, interrupt/timing defects, protocol issues, and toolchain/build differences. Understand its real effect on registers, memory, timing, and hardware rather than memorizing syntax alone.
498	Q: Quel calcul ou contrôle pratique faut-il savoir faire avec senior PIC RCA ? R: Traduire le concept en valeurs de registres, adresses, cycles d'instruction, masques de bits, timing, tension/code ou états protocole. Utiliser le datasheet exact dès qu'une formule périphérique ou un champ registre intervient.	Q: What calculation or practical check should you know for senior PIC RCA? A: Translate the concept into register values, addresses, instruction cycles, bit masks, timing, voltage/code, or protocol states. Use the exact device datasheet whenever a peripheral formula or register field is involved.
499	Q: Comment valider senior PIC RCA sur une vraie carte PIC ? R: Commencer par un firmware minimal, lire le datasheet et vérifier les SFR/config bits. Observer ensuite le comportement avec debugger, oscilloscope ou analyseur logique et comparer les mesures aux cycles/timings calculés.	Q: How would you validate senior PIC RCA on a real PIC board? A: Start with minimal firmware, read the datasheet, and verify SFRs/config bits. Then observe behavior with the debugger, oscilloscope, or logic analyzer and compare measurements with calculated cycles/timing.
500	Q: Scénario Senior : le firmware PIC fonctionne parfois mais échoue après une modification autour de senior PIC RCA. Comment faites-vous le RCA ? R: Reproduire avec un cas minimal et geler la version hardware/toolchain. Vérifier alimentation, reset, clock et config bits, puis registres, banques/adresses, flags, pile, interruptions et timing. Mesurer les signaux externes, comparer le listing/map/HEX si le build a changé, localiser la première divergence et ajouter un test de régression.	Q: Senior scenario: PIC firmware works intermittently but fails after a change involving senior PIC RCA. How do you perform RCA? A: Reproduce with a minimal case and freeze the hardware/toolchain version. Check power, reset, clock, and configuration bits, then registers, banks/addresses, flags, stack, interrupts, and timing. Measure external signals, compare listing/map/HEX if the build changed, locate the first divergence, and add a regression test.

Instruction & Code Quick Reference / Référence rapide

Topic / Sujet	Example / Key point
Literal → W	MOVLW 0x55
W → register	MOVWF target
Set bit	BSF register, bit
Clear bit	BCF register, bit
Test/skip	BTFSS register, bit / BTFSC register, bit
Call/return	CALL routine / RETURN
Interrupt return	RETFIE
Unconditional branch	GOTO label
Classic timing	$F_{cy} = F_{osc}/4$; $T_{cy} = 4/F_{osc}$
16-bit overflow preload	Preload≈65536-N counts
ADC estimate	$V_{in} = Code \times V_{ref} / FullScaleCode$
PWM	$Duty \approx T_{on} / T_{period} \times 100\%$
UART	Calculate BRG value → actual baud → % error
I ² C	Verify ACK/NACK, pull-ups, rise time, bus capacitance, and device addressing
PIC-AS	Current XC8 PIC assembler command-line driver: pic-as

Abbreviations / Abréviations

Abbreviation	Meaning
ALU	Arithmetic Logic Unit
WREG	Working Register
SFR	Special Function Register
GPR	General Purpose Register
FSR	File Select Register
BSR	Bank Select Register
ISR	Interrupt Service Routine
GPIO	General-Purpose Input/Output
ADC	Analog-to-Digital Converter
CCP	Capture/Compare/PWM
PWM	Pulse-Width Modulation
UART	Universal Asynchronous Receiver/Transmitter
SPI	Serial Peripheral Interface
I ² C	Inter-Integrated Circuit
EEPROM	Electrically Erasable Programmable Read-Only Memory
ICSP	In-Circuit Serial Programming
IDE	Integrated Development Environment
RCA	Root Cause Analysis

Senior PIC Assembly Checklist

- Identify the exact PIC core/device before assuming instruction, banking, stack, interrupt, or peripheral behavior.
- Convert Fosc to instruction timing and count cycles accurately, including branch/skip/call/return effects.
- Understand WREG, STATUS, SFR/GPR memory, banking, BSR/access bank, and FSR indirect addressing.



AHDS

Annissoff Hardware Development System
Design once. Reuse forever.

Email: ahds@engineer.com

- Read and write bit masks confidently; understand carry/borrow and multi-byte arithmetic.
- Configure GPIO completely, including direction, latch/PORT behavior and analog multiplexing.
- Design short deterministic ISRs and understand context, flags, latency and interrupt-source clearing.
- Calculate timer preload/prescaler, UART baud error, PWM timing and ADC scale from the device datasheet.
- Validate UART/SPI/I²C/PWM with a scope or logic analyzer, not only with debugger register views.
- Understand EEPROM/Flash endurance and protected write sequences.
- Be able to read legacy MPASM code and migrate it cautiously to MPLAB XC8 PIC Assembler/pic-as.
- During RCA, separate hardware, clock/reset/config bits, addressing/banking, timing/interrupts, protocol and toolchain/build causes.